류현식 | Frontend Developer

지속 가능한 서비스를 고민하는 프론트엔드 개발자

010-7316-7616

rhsik2000@gmail.com

https://www.hyunsik2000.com/

https://github.com/hyunsik2000

https://velog.io/@hyunsik2000/posts

사용자가 더 안정적이고 일관된 경험으로 서비스를 이용 할 수 있도록 돕는 **프론트엔드 개발자**를 목표로 하고 있습니다. 개발은 단순히 기능을 구현하는 일이 아니라, **서비스 구조를 설계하고 사용자 경험을 개선하는 과정**이라고 생각합니다.

기능 구현만큼 "왜 이 구조인가"를 먼저 고민하는 데 시간을 투자합니다 . 팀장 경험을 통해 프로젝트인증 구조, UX 흐름, 공통 컴포넌트 설계를 맡으며 팀원이 일관된 방식으로 개발할 수 있는 환경을 만들어왔습니다. 설계가 바뀌어야 할 때는 과감하게 뒤집고 재설계한 경험이 있습니다. 완성보다 올바른 방향을 더 중요하게 생각합니다.

기술 스택

Frontend	• **Core : Next.js, React , TypeScript, JavaScript** • **Styling : Tailwind CSS** • **State & Data: Zustand**
Infra & Deployment	• **Cloud: AWS EC2, S3** • **CI/CD : GitHub Actions, CodeDeploy** • **Deployment: Vercel**
Tools	• **Git, Github, Figma, Notion, Postman**

교육 및 활동

오즈코딩스쿨 초격차캠프 프론트엔드 15기 수료

2025.09 ~ 2026.03

금오공과대학교 , 컴퓨터공학과 학사 졸업

2019.03 ~ 2025.08

패스트캠퍼스 인터랙티브 웹 퍼블리싱 교육 수료

2024.11 ~ 2025.01

멋쟁이사자처럼 12기 프론트엔드

2024.03 ~ 2024.09

프로젝트

copymark - npm 라이브러리

개인 | 2026.01 ~

copymark/@lyu_danny

NPM Library - ClipBoard 복사 후 Toast 알림

기술 스택

React, Vite, TypeScript

Github

https://github.com/hyunsik2000/copymark

- npm 패키지 배포 및 모노레포 구조 설계
  라이브러리와 데모를 별도 레포로 관리하면 변경마다 재배포가 필요. 모노레포 구조로 통합해 수정사항을 데모에 즉시 반영
- Lazy DOM 주입 패턴 적용
  첫 복사 시점에 createRoot로 Toast를 body에 자동 삽입 → 사용자는 별도 Provider 설정없이 Toast 효과로 결과 확인
- 누적 다운로드 약 300회 달성
  npm 사이트 기준 누적 다운로드 300회 달성. 외부 사용자가 실제로 쓸수 있는 라이브러리 형태를 설계 및 배포

Ozent - 사용자 맞춤 향조 추천 서비스

OZ | 2026.02 ~ 2026.03

관리자 페이지 개발

FE 팀장 FE - 3인 / BE - 3인

기술 스택

Next.js, React, Typescript, TailwindCSS
AWS S3 EC2

Web

https://oz-scent-match.duckdns.org/

Github

https://github.com/team-201-created/FE

- AWS EC2 배포
  EC2 + GitHub Actions + CodeDeploy로 자동 배포 파이프라인 구축, 수동 배포 → 자동화로 전환, 배포 누락 리스크 제거 및 스프린트 집중도 향상
- 캐시 무효화 설계
  어드민 데이터 특성상 캐시 잔류 시 오작동 가능성이 있어 Server Component에서 매 진입 시 fetch + 뮤테이션 직후 캐시 무효화로 항상 최신 상태 보장
- 클라이언트 인증 관리
  매 API 호출마다 토큰 주입·에러 처리 코드가 중복되는 문제를 해결하고자 createFetch 함수를 만들어 공통 로직을 인터셉터에 위임. 인증이 필요한 요청은 서버 컨텍스트에서만 httpOnly 쿠키의 토큰을 읽어야 했기 때문에, dynamic import로 처리해 클라이언트 환경 오염을 방지
- 전역 모달 시스템 설계
  Zustand 기반 전역 Store로 모달 상태를 통합 관리하도록 구조 개선 React Portal과 배열 스택 방식으로 DOM 최상위에서 모달 흐름 제어
- Next 환경에서 SEO 작업
  SSR 렌더링시 Next.js metadata, sitemap, robots 설정을 통한 SEO 최적화 적용 후 Lighthouse SEO 테스트 82점 → 100점으로 개선. 실제 서비스로서의 지표 확인을 위해 Google Search Console 색인 등록 후 검색 노출 및 클릭 수 지표 확인

LMS 통합 관리 시스템

OZ Externship | 2026.01 ~ 2026.02

쪽지시험 관리 페이지 개발

FE 팀장 FE - 3인 / BE - 3인

기술 스택

React, Vite, TypeScript, TailwindCSS
Axios, Zustand

Github

https://github.com/OZ-Coding-School/oz_externship_fe_06_team1

- 전역 에러 핸들링 로직 설계
  공통 Axios 인스턴스와 API 훅을 설계하여, 팀원에게 로딩 상태와 전역적 에러 처리를 자동적으로 처리 할 수 있도록 표준화 후 지역적 에러는 객체 상태로 받아 커스텀 할 수 있는 구조로 개선.
  - 초기 설계에서 모든 에러를 Axios interceptor → 전역 스토어로 전파하는 중앙집권식 구조를 적용했으나 로그인 폼처럼 지역적으로 따로 처리해야 하는 케이스에서 한계를 발견.
  - 지역 처리 우선 → 미처리 에러만 전역 전파하는 계층형 구조로 재설계 후 useAxios 훅의 onError 콜백이 true를 반환하면 전역 스토어 전파를 차단하고, ErrorBoundary를 추가해 예상치 못한 런타임 에러까지 3계층으로 방어.
- 범용 데이터 테이블 컴포넌트 설계
  페이지마다 반복 구현되던 테이블 UI를 복합 컴포넌트 패턴으로 추상화. 사용 가이드 문서와 함께 팀 내 공유 → 팀원 피드백 기준 반복 작업 체감 감소
- 코드 품질 관리 자동화
  Husky와 lint-staged를 도입하여 커밋 시 자동 린트 검사 프로세스를 구축하여 리뷰 사이클에서 코드와 컨벤션 스타일 관련 피드백 제거, 코드 리뷰에 집중할 수 있는 환경 구축

Fizz! - Wellness 챌린지 숏폼 플랫폼

멋쟁이 사자처럼 | 2024.07 ~ 2024.08

영상 스트리밍 및 업로드 전담 개발

FE 팀장 FE - 3인 / BE - 3인

기술 스택

React, JavaScript, TailwindCSS, SSE

Github

https://github.com/Fizz-challenge/Fizz_front

- hls.js 기반 스트리밍 최적화
  영상을 다룰 때 일반 <video> 태그로는 m3u8 형식의 청크 분할 스트리밍이 불가능해 hls.js 도입. 버퍼 크기 조정하여 재생까지의 지연 시간을 단축
- 메모리 누수 방지 및 성능 관리
  쇼츠 특성상 사용자가 영상을 빠르게 넘기며 hls 인스턴스가 누적되는 문제 발생. Intersection Observer API와 결합해 화면에 보이는 영상에만 hls 인스턴스를 활성화하고, 벗어난 영상은 즉시 리소스 해제 → 장시간 스크롤 시 메모리 누적 방지
- 영상 업로드 구조 설계
  서버를 거쳐 업로드 시 병목이 발생하는 구조를 개선하기 위해 AWS S3 Presigned URL + Multipart Upload 방식으로 클라이언트 직접 업로드 구현. SSE로 업로드 진행률을 실시간 표시해 사용자 이탈 방지

CaféFlow - 실시간 카페 좌석 현황 서비스

학과 졸업작품 | 2024.03 ~ 2024.06

Map 및 실시간 통신 구조 개발

FE 팀장 FE - 3인 / BE - 3인

기술 스택

React, JavaScript, Naver Map API, SocketJS

Github

https://github.com/Cafe-Flow/CafeFlow-FrontEnd

- Naver Map API + 실시간 혼잡도 시각화
  REST 폴링으로는 혼잡도 변경을 즉시 반영하기 어렵고 불필요한 요청이 반복되는 문제 발생. STOMP + SocketJS로 클라이언트별 Subscribe 객체를 유지해 혼잡도 변경 시에만 데이터를 수신 → 마커 색상(원활/혼잡)을 실시간 업데이트
- 1:1 채팅 기능 구현
  사용자가 채팅을 요청할 때만 채팅 전용 STOMP 클라이언트 객체를 생성해 Subscribe/Publish 연결. 카페 관리자-사용자 간 실시간 상담 지원
- 전역 상태 설계 및 재사용 컴포넌트
  사용자 정보를 여러 컴포넌트에서 props로 반복 전달하는 drilling 문제를 Context API로 중앙화. 커뮤니티 게시판 테이블도 재사용 컴포넌트로 추상화해 중복 구현 제거

자격

공인 TOEIC 725

2025.02

정보처리기사

2024.12